

Projeto MecaniQA

Autores:

Gustavo Prates Caetano

Matheus Costa Rodrigues

Nallanda Novato

Thainá Araujo Salgado

Vinicius Martins Lebrão

Vitor Hugo Lima Silva Bittencourt

Agenda

Apresentar a evolução estrutural da API RESTful, incorporando novos padrões de projeto e isolamento de domínio, cumprindo os requisitos da OAT 2.

1. Expansão do Domínio

4. Isolamento com DTOs

2. Entidade Associativa

5. Fluxo Lógico
(Diagrama de Atividade)

3. O Padrão Builder

Expansão do Domínio



- Novas Entidades Core

Para suportar o fluxo completo da oficina, incluímos o rastreamento de serviços prestados e a gestão de suprimentos.

- Ordem de Serviço (OS)

Agrupa os serviços prestados e controla o andamento através de um ciclo de vida rigoroso.

- Pedido de Peças

Gerencia o processo de requisição de materiais necessários para a execução dos serviços.

Entidade Associativa



- Resolução N:N
- Criação da ItemPedidoPeca.
- Armazena a quantidade exata de itens.
- Permite múltiplas peças iguais no mesmo pedido.

Padrão Builder



- Builder obrigatório na construção de OrdemServico.
- Implementação manual, com métodos encadeados retornando this.
- Construtor de OrdemServico privado e recebendo o Builder.
- A criação direta com new OrdemServico() deve falhar; a construção ocorre por OrdemServico.builder().

```
package br.com.mecaniQA.api;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class OrdemServico { 16 usages  🧑 Gustavo Prates Caetano
    private Long codigoUnico; 3 usages
    private String placaVeiculo; 3 usages
    private String descricaoProblema; 3 usages
    private StatusOS status; 3 usages
    private List<Servico> servicos; 3 usages
    private List<ItemPedidoPeca> pecasUtilizadas; 3 usages
    private LocalDateTime dataAbertura; 3 usages

    private OrdemServico(OrdemServicoBuilder builder) { 1 us
        this.codigoUnico = builder.codigoUnico;
        this.placaVeiculo = builder.placaVeiculo;
        this.descricaoProblema = builder.descricaoProblema;
        this.status = builder.status;
        this.servicos = builder.servicos;
```

Isolamento de Domínio (DTO)

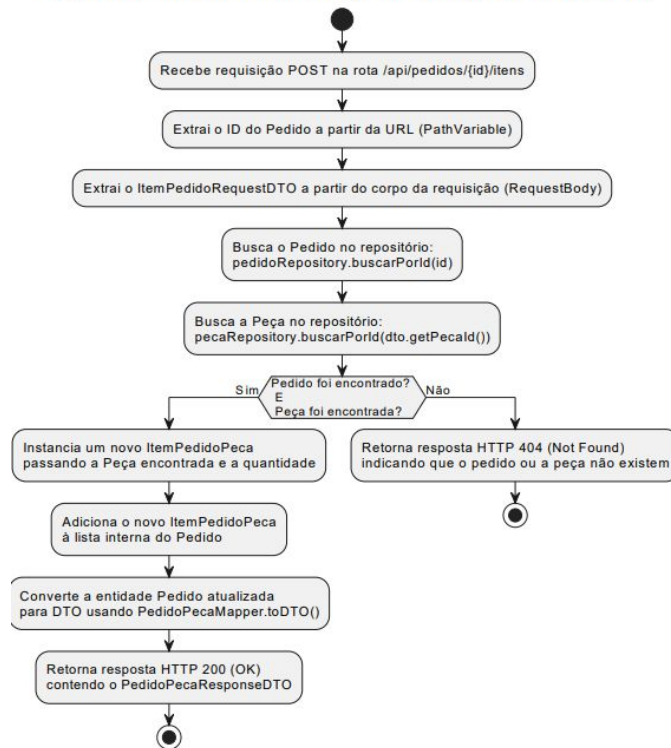


- As Entidades puras não devem trafegar nos endpoints.
- Utilizamos Data Transfer Objects (DTO) para controlar a carga útil.
- JSON Cliente -> OrdemServicoDTO -> Entidade OS
- Mappers: Classes responsáveis por converter Entity em DTO e vice-versa, mantendo os Controllers limpos.

Fluxo Lógico (Diagrama de Atividade)

- A visão do Controller garante a integridade da requisição antes de persistir os dados.
- Recebe Requisição (DTO).
- Valida dados obrigatórios (Caminho de Erro - 400).
- Aciona o Mapper e constrói via Builder.
- Retorna 201 Created (Caminho de Sucesso).

Diagrama de Atividade: Adicionar Peça ao Pedido (Visão do Controller)



Referências



BLOCH, Joshua. **Java efetivo**. 3. ed. Rio de Janeiro: Alta Books, 2019.

PLANTUML. **PlantUML language reference guide**. [S. l.]: PlantUML, 2026. Disponível em: <https://plantuml.com/>. Acesso em: 28 set. 2026.

VALENTE, Marco Tulio. **Engenharia de software moderna**: princípios e práticas para desenvolvimento de software com produtividade. Belo Horizonte: [s. n.], 2020.